

Common Instructions: For this assignment you will need to write a program to solve a problem, based on handout code provided on D2L. You should upload your solution to D2L in individual plaintext files with the correct extension for source code (i.e. `.py` for Python, `.c` or `.h` for C). In general, you do not need to submit handout code files that are unchanged; you only need to upload code files that you modified in some way.

Project Description: Many of the OS algorithms we will study in this class rely on data structures such as linked lists, queues, and hashtables. However, the C programming language (in which all operating systems are written) does not include any data structures in its standard library; operating system writers must create them from scratch before they can use them. To give you some experience with this process, in this problem you will implement a basic queue in C. The queue will store untyped pointers (`void*`) to allow it to store any data type.

The compressed folder `queue-cmake-project.zip`, available on D2L, contains 3 code files: `queue.h` provides the interface for the queue, `queue.c` provides the implementation of the queue functions, and `test_queue.c` tests the queue and provides the `main` function. It also contains a file named `README.md` that describes how to compile and run the project, either with or without CMake.

You must write all of the (initially-empty) functions in `queue.c`, and provide a definition of the `struct queue` structure; you may also define any other structures or functions you want to use to support your implementation inside `queue.c`. Make sure your functions adhere to the interface described in the documentation comments in `queue.h`, and do not modify `queue.h`. You must also write additional test cases in `test_queue.c` to ensure you have tested all of the queue's functions and tested them in all of the edge cases that may arise. The handout version of `test_queue.c` is very basic and just shows how the queue might be used in one case.